

AI 에이전트

전문가 양성 과정

함수



Contents

1. 함수

- 함수 기초
- 가변 인자
- Scope
- 함수 응용
- 내장 함수
- lambda

2. 모듈과 패키지

- 모듈
- Python 기본 모듈
- 패키지

3. 참고 자료

- 타입 힌팅
- 데코레이터

제 1장.

함수

1-1 함수 기초

함수 기초

- 자주 사용하는 코드를 이름 붙여 묶어둔 것

```
stock_name_1 = "삼성전자"  
current_price_1 = 73500  
buy_price_1 = 70000  
quantity_1 = 10
```

```
profit_amount_1 = (current_price_1 - buy_price_1) * quantity_1  
print(f"{stock_name_1} 평가 손익: {profit_amount_1}원")
```

```
stock_name_2 = "SK하이닉스"  
current_price_2 = 118000  
buy_price_2 = 120000  
quantity_2 = 3
```

```
profit_amount_2 = (current_price_2 - buy_price_2) * quantity_2  
print(f"{stock_name_2} 평가 손익: {profit_amount_2}원")
```

함수 기초

- 자주 사용하는 코드를 이름 붙여 묶어둔 것

```
def show_profit_message(stock_name, buy_price, current_price, quantity):  
    profit_amount = (current_price - buy_price) * quantity  
    print(f"{stock_name} 평가 손익: {profit_amount}원")
```

```
show_profit_message("삼성전자", 70000, 73500, 10)  
show_profit_message("SK하이닉스", 120000, 118000, 3)
```

함수 기초

- 자주 사용하는 코드를 이름 붙여 묶어둔 것

```
print("hello world")
```

```
len([1, 2, 3, 4, 5])
```

함수 기초

- 자주 사용하는 코드를 이름 붙여 묶어둔 것

len([1, 2, 3, 4, 5])



```
numbers = [1, 2, 3, 4, 5]
```

```
count = 0
```

```
for i in numbers:
```

```
    count += 1
```

```
print(count)
```


함수 기초

- 자주 사용하는 코드를 이름 붙여 묶어둔 것
 - 프로그래밍을 하다 보면 똑같은 내용을 반복해서 작성하는 경우 발생
 - 반복되는 부분이 있을 경우, '반복적으로 사용되는 가치 있는 부분'을 한 뭉치로 묶어 함수로 작성
- 함수 활용의 장점
 - 필요할 때마다 함수 이름을 불러서 실행할 수 있음
 - 같은 코드를 반복해서 쓰지 않아도 됨
 - 프로그램을 작은 단위로 나누어 관리할 수 있음

```
print("안녕하세요!")  
print("환영합니다.")
```

```
def say_hello():  
    print("안녕하세요!")  
    print("환영합니다.")
```

```
say_hello()  
say_hello()
```

함수 기초

- 함수의 구조
 - def는 함수를 만들 때 사용하는 키워드
 - 함수 이름 뒤에는 소괄호 ()와 콜론 : 을 붙인다
 - 함수 안에서 실행할 코드는 들여쓰기
 - 매개변수 (parameter)
 - : 함수가 값을 받을 때 사용하는 변수
 - 인자 (argument)
 - : 함수를 호출할 때 실제로 전달하는 값

```
def 함수_이름(매개변수):  
    수행할_코드1  
    수행할_코드2  
    ...
```

```
def add(a, b):  
    print(a + b)
```

함수 기초

- 함수의 구조
 - 함수는 만들어도 자동 실행되지 않음, 반드시 호출(실행) 코드가 있어야 함

```
# 함수 작성
def add(a, b):
    print(a + b)

# 함수 호출
add(5, 3)      # 8
add(12, 13)   # 25
```

함수 기초

- 아래 코드 실행 결과는?

```
def add(a, b):  
    print(a + b)  
  
print("함수 호출 전")  
  
add(15, 13)  
  
print("함수 호출 후")
```

함수 기초

- **return**
 - 함수의 결과를 함수 밖으로 반환하는 문법
 - return하지 않은 함수는 None 반환

```
import keyword
print(keyword.kwlist)

"""
['False', 'None', 'True', 'and', 'as', 'assert',
'async', 'await', 'break', 'class', 'continue', 'def',
'del', 'elif', 'else', 'except', 'finally', 'for',
'from', 'global', 'if', 'import', 'in', 'is', 'lambda',
'nonlocal', 'not', 'or', 'pass', 'raise', 'return',
'try', 'while', 'with', 'yield']
"""
```

```
def add(a, b):
    return a + b

total = add(10, 20)

print(total) # 30
```

함수 기초

- `return`
 - 함수의 결과를 함수 밖으로 반환하는 문법
 - * `print` : 터미널에 출력하기 위한 함수

```
def add(a, b):  
    return a + b  
  
total = add(10, 20)  
  
print(total) # 30
```

```
def add(a, b):  
    print(a + b)  
  
total = add(10, 20)  
  
print(total) # None
```

👉 `return`을 사용하면 결과를 다시 사용 가능

함수 기초

- `return`
 - `return`이 실행되면 함수는 즉시 종료
 - `return` 아래에 있는 코드는 실행되지 않음

```
def get_result(a, b):  
    return a + b  
    return a * b  
  
result = get_result(3, 5)  
  
print(result) # 8
```

👉 여러 값을 반환하고 싶다면?

함수 기초

- `return`
 - `return` 을 여러 번 쓰는 것이 아니라, 한 번의 `return` 으로 여러 값을 함께 반환
 - 튜플로 묶여서 반환

```
def get_result(a, b):  
    return a + b  
    return a * b  
  
result = get_result(3, 5)  
  
print(result) # 8
```

```
def get_result(a, b):  
    return a + b , a * b  
  
result = get_result(3, 5)  
  
print(result) # (8, 15)
```


함수 기초

- 파라미터가 없는 함수
 - 파라미터가 없는 함수가 가능할까?

```
def say():  
    return 'Hi'
```

함수 기초

- 위치 인자 vs 키워드 인자
 - 함수 호출 시 인자는 위치에 따라 함수에 전달 (위치 인자)
 - 키워드 인자 : 변수 이름으로 특정 인자를 전달
 - 어떤 값이 어떤 매개변수에 들어가는지 명확
 - 인자의 순서를 덜 헷갈림

```
def say_hello(name, age, language):  
    print(name)  
    print(age)  
    print(language)  
  
say_hello("Jay", 25, "Python")  
say_hello(name="Jay", age=25, language="Python")  
say_hello(language="Python", name="Jay", age=25)
```

함수 기초

- 위치 인자 vs 키워드 인자
 - 키워드 인자 다음 위치 인자를 활용할 순 없음

```
def say_hello(name, age, language):  
    print(name)  
    print(age)  
    print(language)  
  
say_hello(name="Jay", 25, "Python")
```

👉 에러 발생

함수 기초

- 기본값이 있는 파라미터
 - 매개변수에 기본값을 줄 수 있음
 - 값을 전달하지 않으면 기본값 사용 / 값을 전달하면 전달한 값을 사용

```
def say_hello(name="손님"):
    print(f"안녕하세요, {name}님!")

say_hello()
say_hello("Jay")
```

함수 기초

- 실행될까? 결과는 어떻게 출력될까?

```
def say_hello(name, age = 30, language = "Python"):
    print(name)
    print(age)
    print(language)

say_hello(name="Jay")
say_hello(name="Amy", age=25)
```

1-2 가변 인자

가변 인자

- 함수에 전달할 값의 개수가 매번 달라질 수 있음
- 이때 사용하는 문법이 가변 인자
- *args, **kwargs 를 주로 사용

```
print('hello', 'world')
```

가변 인자

- `*args` : 여러 개의 위치 인자를 받을 때 사용
 - 일반 함수는 인자 개수가 정해져 있음
 - 숫자 2개가 아니라 3개, 4개, 5개를 더하고 싶다면?
 - 매번 함수를 새로 만들 수는 없음

```
def add(a, b):  
    return a + b  
  
print(add(10, 20))
```

```
add(10, 20, 30)  
add(10, 20, 30, 40)  
add(10, 20, 30, 40, 50)
```


가변 인자

- *args : 여러 개의 위치 인자를 받을 때 사용
 - 입력 파라미터 앞에 asterisk(*) 추가

```
def add(a, b):  
    return a + b  
  
print(add(10, 20))
```

```
def add(*args) :  
    total = 0  
    for i in args :  
        total += i  
    return total  
  
print(add(10, 20))      # 30  
  
add(10, 20, 30)         # 60  
add(10, 20, 30, 40)     # 100  
add(10, 20, 30, 40, 50) # 150
```

가변 인자

- 꼭 args라는 이름을 써야 하는 것은 아님
 - 하지만 실무에서는 *args라는 이름을 많이 사용
- 튜플로 반환

```
def print_profile(name, age, *languages):  
    print(name)  
    print(age)  
    print(languages)  
    for language in languages:  
        print(language)
```

```
print_profile('Jay', 30, 'Python', 'JavaScript', 'C++')
```

가변 인자

- `**kwargs` : 여러 개의 키워드 인자를 받을 때 사용
- 딕셔너리로 반환

```
def print_user_info(**kwargs):  
    print(kwargs)  
  
print_user_info(name="Jay", age=30, job="Student")  
# {'name': 'Jay', 'age': 30, 'job': 'Student'}
```

가변 인자

- `**kwargs` : 여러 개의 키워드 인자를 받을 때 사용
- 딕셔너리로 반환 → `get()` 함수로 안전하게 조회

```
def print_user_info(**kwargs):  
    name = kwargs.get("name")  
    age = kwargs.get("age")  
    job = kwargs.get("job")  
  
    print(f"이름: {name}")  
    print(f"나이: {age}")  
    print(f"직업: {job}")  
  
print_user_info(name="Jay", age=30, job="Student")
```

가변 인자

- Asterisk *

1. 함수를 정의할 때 *를 붙이면 여러 인자를 받을 수 있음

```
def add(*args):  
    return sum(args)  
  
print(add(10, 2, 3))    # 15
```

```
print(*objects, sep=' ', end='\n', file=None, flush=False)
```

Print *objects* to the text stream *file*, separated by *sep* and followed by *end*. *sep*, *end*, *file*, and *flush*, if present, must be given as keyword arguments.

모든 비 키워드 인자는 [str\(\)](#) 이 하듯이 문자열로 변환된 후 스트림에 쓰이는데, *sep* 로 구분되고 *end* 를 뒤에 붙입니다. *sep* 과 *end* 는 모두 문자열이어야 합니다; *None* 일 수도 있는데, 기본값을 사용한다는 뜻입니다. *objects* 가 주어지지 않으면 [print\(\)](#) 는 *end* 만 씁니다.

file 인자는 `write(string)` 메서드를 가진 객체여야 합니다; 존재하지 않거나 *None* 이면, [sys.stdout](#) 이 사용됩니다. 인쇄된 인자는 텍스트 문자열로 변환되기 때문에, [print\(\)](#) 는 바이너리 모드 파일 객체와 함께 사용할 수 없습니다. 이를 위해서는. 대신 `file.write(...)` 를 사용합니다.

Output buffering is usually determined by *file*. However, if *flush* is true, the stream is forcibly flushed.

가변 인자

- Asterisk *
2. 딕셔너리를 키워드 인자로 펼칠 때 ** 사용

```
def introduce(name, age, job):  
    print(f"이름: {name}")  
    print(f"나이: {age}")  
    print(f"직업: {job}")  
  
user = {  
    "name": "Jay",  
    "age": 30,  
    "job": "Student"  
}  
  
introduce(**user)
```

가변 인자

- Asterisk *

- 3. 컨테이너 언패킹 시 * 사용

```
numbers = [1, 2, 3, 4, 5]

a, b, c, d, e = numbers

print(a, b, c, d, e) # 1 2 3 4 5

a, b, *rest = numbers

print(a, b, rest) # 1 2 [3, 4, 5]
```

1-3 Scope

Scope

- Python 함수는 코드 내부와 외부 구분
 - (함수 내부 : local scope / 외부 : gobal scope)
- 변수
 - 전역 변수 (global variable) : global scope(함수 외부)에 정의된 변수
 - 지역 변수 (local variable) : local scope(함수 내부)에 정의된 변수

```
message = '결과 : '
```

```
def add(a, b):  
    result = a + b  
    return result
```

Scope

- 지역 변수와 전역 변수
 - 함수 안에서 만든 변수는 보통 함수 안에서만 사용 가능
 - `result`는 함수 안에서 만든 변수 → 함수 밖에서는 사용할 수 없음 (지역 변수)

```
def add(a, b):  
    result = a + b  
    return result
```

```
print(result) # NameError: name 'result' is not defined
```

Scope

- 지역 변수와 전역 변수
 - 전역 변수 : 함수 밖에서 만든 변수
 - 함수 안에서 읽을 수 있음

```
message = '결과 : '  
  
def add(a, b):  
    result = a + b  
    return message + str(result)  
  
print(add(5, 3))
```

Scope

- 지역 변수와 전역 변수
 - 함수 안에서 message 라는 지역 변수를 새롭게 생성
 - 전역 변수 message에 재할당 x

```
message = '결과 : '  
  
def add(a, b):  
    result = a + b  
    message = "계산 결과는 "  
    print(message)  
    return message + str(result)  
  
print(add(5, 3))  
  
print(message)
```

Scope

- 지역 변수와 전역 변수
 - `global` : 함수 안에서 전역 변수를 수정할 때 사용하는 키워드

```
message = '결과 : '  
  
def add(a, b):  
    global message  
    message = '덧셈 결과 : '  
    result = a + b  
    return message + str(result)  
  
print(add(5, 3))
```

1-4 함수 응용

함수 응용

- docstring
 - 함수에 남기는 설명 문장
 - 함수가 무엇을 하는지
 - 어떤 값을 받는지
 - 무엇을 반환하는지
 - 함수 바로 아래에 삼중 따옴표로 작성

```
def add(a, b):  
    """  
        두 숫자를 더한 결과를 반환한다.  
  
    Args :  
        a : float, int  
        b : float, int  
  
    Returns :  
        float, int : a와 b의 합  
    """  
    return a + b  
  
print(add.__doc__)
```

함수 응용

- docstring
 - 주석 : 코드 중간에 설명을 남길 때 사용
 - docstring : 함수, 클래스, 모듈의 설명을 남길 때 사용
 - 향후 Agent 개발 시 AI가 해당 함수를 호출할 때 참고 자료로 사용

```
def get_weather(location) :  
    """  
    특정 지역의 날씨 정보 제공  
    Args:  
        location: 날씨 정보를 알고 싶은 지역의 이름 (str)  
    Returns:  
        해당 지역의 날씨 정보 (str)  
    """  
    return f"{location}의 날씨는 : 눈, -2°C, 미세먼지 매우나쁨"
```


함수 응용

- type hinting
 - 함수가 어떤 자료형을 받고, 어떤 자료형을 반환하는지 자료형을 힌트로 적어주는 문법
 - 코드 실행을 강제로 막는 기능 x

```
def get_weather(location: str) -> str:
    """
    특정 지역의 날씨 정보 제공
    Args:
        location: 날씨 정보를 알고 싶은 지역의 이름 (str)
    Returns:
        해당 지역의 날씨 정보 (str)
    """
    return f"{location}의 날씨는 : 눈, -2°C, 미세먼지 매우나쁨"

print(get_weather.__annotations__)
```

함수 응용

- type hinting
 - 코드 실행을 강제로 막는 기능 x

```
def add(a: int, b: int) -> int:  
    return a + b  
  
print(add("10", "20")) # 1020
```

👉 타입이 다르더라도 에러 발생 x

함수 응용

- type hinting

자료형	type hint
정수	int
실수	float
문자열	str
Boolean	bool
리스트	list
정수 리스트	list[int]
딕셔너리	dict[str, int]

1-5 내장 함수

내장 함수

- 함수의 종류
 - 내장 함수 : 파이썬에 기본적으로 포함된 함수
 - 외장 함수 : 외부 라이브러리에서 제공하는 함수 (import를 통해 사용)
 - 사용자 정의 함수 : 사용자가 직접 작성한 함수

파이썬 내장 함수

(<https://docs.python.org/ko/3/library/functions.html>)

A abs() aiter() all() anext() any() ascii()	E enumerate() eval() exec()	L len() list() locals()	R range() repr() reversed() round()
B bin() bool() breakpoint() bytearray() bytes()	F filter() float() format() frozenset()	M map() max() memoryview() min()	S set() setattr() slice() sorted() staticmethod() str() sum() super()
C callable() chr() classmethod() compile() complex()	G getattr() globals()	N next()	T tuple() type()
D delattr() dict() dir() divmod()	H hasattr() hash() help() hex()	O object() oct() open() ord()	V vars()
	I id() input() int() isinstance() issubclass() iter()	P pow() print() property()	Z zip()
			- __import__()

(복습) for

- `enumerate()`
 - 리스트를 반복할 때 순서 번호와 값을 함께 꺼내고 싶을 때 사용
 - 기본 인덱스는 0부터 시작, `start=` 를 사용하면 x번부터 출력 가능

```
fruits = ["apple", "banana", "orange"]
```

```
for index, fruit in enumerate(fruits):  
    print(index, fruit)
```

```
# 0 apple  
# 1 banana  
# 2 orange
```

```
fruits = ["apple", "banana", "orange"]
```

```
for index, fruit in enumerate(fruits, start=1):  
    print(f"{index}. {fruit}")
```

```
# 1. apple  
# 2. banana  
# 3. orange
```

(복습) for

- zip()
 - 여러 리스트를 나란히 묶어서 반복할 때 사용

```
names = ["철수", "영희", "민수"]
scores = [90, 85, 70]

for name, score in zip(names, scores):
    print(f"{name}: {score}점")

# 철수: 90점
# 영희: 85점
# 민수: 70점
```

내장 함수

- `map()`
 - 여러 데이터에 같은 함수를 한 번씩 적용할 때 사용
 - `map(함수, 반복 가능한 객체)`
 - `map()`의 결과는 바로 리스트 x, 필요하면 `list()`로 감싸서 확인

```
numbers = [1, 2, 3, 4, 5]
result = []
for number in numbers:
    result.append(number ** 2)
print(result)
```



```
def square(number):
    return number ** 2

numbers = [1, 2, 3, 4, 5]
result = list(map(square, numbers))
print(result)
```


내장 함수

- `filter()`
 - 여러 데이터 중 조건을 만족하는 값만 고를 때 사용
 - `filter(함수, 반복 가능한 객체)`

```
numbers = [1, 2, 3, 4, 5]

even_numbers = []

for number in numbers:
    if number % 2 == 0:
        even_numbers.append(number)

print(even_numbers)
```



```
def is_even(number):
    return number % 2 == 0

numbers = [1, 2, 3, 4, 5]

result = list(filter(is_even, numbers))

print(result)
```

1-6 lambda

lambda

- lambda : 짧게 쓰는 이름 없는 함수
 - 일반 함수처럼 def를 쓰지 않음 (보통 한 줄로 간단한 기능을 표현할 때 사용)

```
def add(a, b):  
    return a + b
```

→

```
add = lambda a, b: a + b
```

lambda

- lambda : 짧게 쓰는 이름 없는 함수
 - 일반 함수처럼 def를 쓰지 않음 (보통 한 줄로 간단한 기능을 표현할 때 사용)

lambda 매개변수 1, 매개변수 2 : 반환값

```
add = lambda a, b: a + b
```

lambda

- lambda와 내장함수
 - 데이터 전처리 시 자주 lambda와 내장함수를 활용

```
prices = ["12,000", "3,500", "25,000", "900"]  
  
cleaned_prices = list(map(lambda price: int(price.replace(",", "")), prices))  
  
print(cleaned_prices) # [12000, 3500, 25000, 900]
```

lambda

- lambda와 내장함수
 - 데이터 전처리 시 자주 lambda와 내장함수를 활용
- map()

```
def dollar_to_won(price):  
    return price * 1420  
  
dollar_prices = [10, 20, 35, 50, 100]  
  
result = list(map(dollar_to_won, dollar_prices))  
  
print(result) # [14200, 28400, 49700, 71000, 142000]
```

```
dollar_prices = [10, 20, 35, 50, 100]  
  
result = list(map(lambda price: price * 1420, dollar_prices))  
  
print(result) # [14200, 28400, 49700, 71000, 142000]
```

lambda

- lambda와 내장함수
 - 데이터 전처리 시 자주 lambda와 내장함수를 활용
- filter()

```
stocks = [  
    {"name": "삼성전자", "return_rate": 3.2},  
    {"name": "카카오", "return_rate": -5.1},  
    {"name": "네이버", "return_rate": 1.8},  
    {"name": "LG에너지솔루션", "return_rate": -2.4},  
    {"name": "현대차", "return_rate": 4.5},  
]  
  
profit_stocks = list(filter(lambda stock: stock["return_rate"] > 0, stocks))  
  
for stock in profit_stocks:  
    print(f"{stock['name']}: 수익률 {stock['return_rate']}%")
```

제 2장.

모듈과 패키지

2-1 모듈

모듈

- 모듈러 건축



모듈

- 레고 모듈

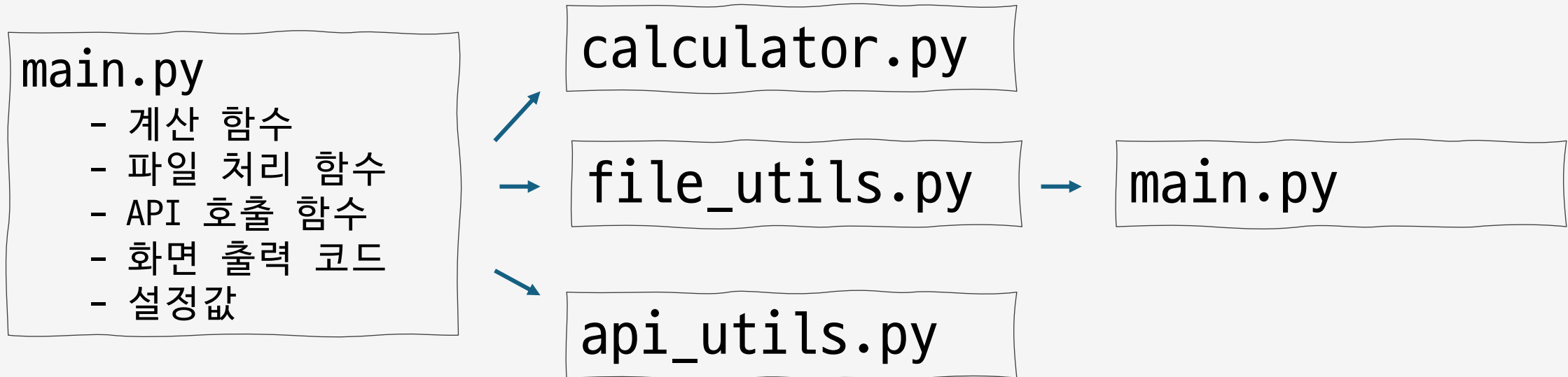


모듈

관련된 함수, 변수, 클래스를 모아둔 Python 파일

모듈

- 모듈의 이점



👉 코드를 역할별로 나눠, 재사용 및 관리 용이

모듈

- 모듈 작성 및 불러오기
 - calculator.py 파일 자체가 calculator 모듈
 - main.py에서 calculator.py 를 불러와 사용

```
# calculator.py
```

```
def add(a, b):  
    return a + b
```

```
def subtract(a, b):  
    return a - b
```

```
# main.py  
import calculator
```

```
result1 = calculator.add(10, 20)  
result2 = calculator.subtract(20, 10)
```

```
print(result1)  
print(result2)
```

모듈

- 특정 함수만 가져오기
 - from 모듈명 import 함수

```
# main.py
import calculator

result1 = calculator.add(10, 20)
result2 = calculator.subtract(20, 10)

print(result1)
print(result2)
```

```
# main.py
from calculator import add

result1 = add(10, 20)

print(result1)
```


모듈

- 특정 함수만 가져오기
 - from 모듈명 import * 이렇게 작성하면 어떻게 될까?

```
# main.py
from calculator import *

result1 = add(10, 20)

print(result1)
```


모듈

- **as**
 - 모듈 이름이 길거나 자주 사용할 때는 별명을 붙이는 문법
 - `import 모듈명 as 별명`

```
# main.py
import calculator as calc

result1 = calc.add(10, 20)
result2 = calc.subtract(20, 10)

print(result1)
print(result2)
```

```
import pandas as pd
import numpy as np
```

모듈

- 엔트리 포인트
 - 아래 코드 실행 결과는?

```
# calculator.py

def add(a, b):
    return a + b

def subtract(a, b):
    return a - b

# 테스트 코드 작성
print(add(10, 20))
```

```
# main.py
import calculator

result = calculator.add(20, 30)

print(result)
```

모듈

- 엔트리 포인트
 - 프로그램 실행이 시작되는 지점
 - 이 파일을 직접 실행했을 때 어디부터 시작할지 정해주는 코드
 - main.py에서 import만 해도 print 코드가 실행

```
# calculator.py
```

```
def add(a, b):  
    return a + b
```

```
def subtract(a, b):  
    return a - b
```

```
# 테스트 코드 작성  
print(add(10, 20))
```

```
# main.py  
import calculator
```

모듈

- 엔트리 포인트
 - `if __name__ == "__main__"`
 - 파이썬이 내부적으로 사용하는 특별한 변수 이름
 - `calculator.py` 파일을 실행할 경우, `calculator.py` 의 `__name__` 변수에는 `__main__` 값이 저장
 - 하지만 다른 파이썬 파일에서 `calculator`를 `import`할 경우
`calculator.py` 의 `__name__` 변수에 `calculator.py` 의 모듈 이름인 `calculator` 가 저장

```
# calculator.py
def add(a, b):
    return a + b

if __name__ == "__main__":
    print(add(10, 20))
```

모듈

- 엔트리 포인트
 - calculator.py를 직접 실행할 때만 아래 코드 실행, 다른 파일에서 import할 땐 실행 x

```
# calculator.py

def add(a, b):
    return a + b

def subtract(a, b):
    return a - b

if __name__ == "__main__":
    print(add(10, 20))
```

모듈

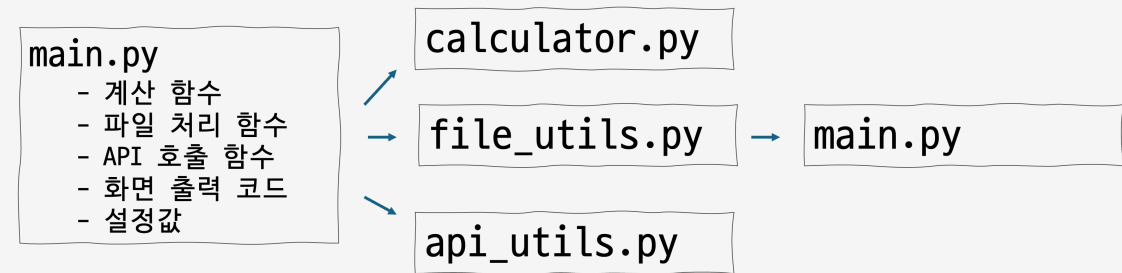
- `_함수`
 - 내부용 함수라는 것을 표현하는 관례
 - 강제는 아니지만 외부에서 직접 쓰지 않는 것이 좋음

```
# calculator.py
def add(a, b):
    return a + b

def _helper():
    pass
```

모듈

- 모듈의 장점
 - 코드가 짧아짐
 - 역할별로 파일을 관리할 수 있음
 - 같은 함수를 여러 파일에서 재사용할 수 있음
 - 오류가 났을 때 찾기 쉬움
 - 협업할 때 파일별로 작업을 나누기 좋음



2-2 Python 기본 모듈

Python 기본 모듈

- <https://docs.python.org/3/py-modindex.html>

<code>__future__</code>	Future statement definitions
<code>__main__</code>	The environment where top-level code is run. Covers command-line interfaces, import-time behavior, and <code>__name__ == '__main__'</code> .
<code>_thread</code>	Low-level threading API.
<code>_tkinter</code>	A binary module that contains the low-level interface to Tcl/Tk.
a	
<code>abc</code>	Abstract base classes according to :pep: `3119`.
<code>aifc</code>	Deprecated: Removed in 3.13.
<code>annotationlib</code>	Functionality for introspecting annotations
<code>argparse</code>	Command-line option and argument parsing library.
<code>array</code>	Space efficient arrays of uniformly typed numeric values.
<code>ast</code>	Abstract Syntax Tree classes and manipulation.
<code>asynchat</code>	Deprecated: Removed in 3.12.
<code>asyncio</code>	Asynchronous I/O.
<code>asyncore</code>	Deprecated: Removed in 3.12.
<code>atexit</code>	Register and execute cleanup functions.
<code>audioop</code>	Deprecated: Removed in 3.13.

모듈	용도
math	수학 계산
random	랜덤 값 생성
datetime	날짜와 시간 처리
os	운영체제 기능 사용
json	JSON 데이터 처리
pathlib	파일, 폴더의 경로를 객체로 처리

(복습) 자료형

- 실수 (float) : 소수점이 있는 숫자
 - math라는 모듈 사용해 확인

```
import math

temperature = 36.5
temperature2 = 36.4

diff = temperature - temperature2

print(diff)
# 0.1000000000000000142

print(math.isclose(diff, 0.1))
# True
```

Python 기본 모듈

- math
 - sqrt() : 제곱근
 - pi : 원주율
 - ceil() : 올림
 - floor() : 내림

```
import math

print(math.sqrt(16))    # 4.0
print(math.pi)         # 3.141592653589793
print(math.ceil(3.2))   # 4
print(math.floor(3.8))  # 3
```

Python 기본 모듈

- datetime
 - 현재 날짜와 시간, 연도, 월, 일 같은 값 가져오기
 - 첫 번째 datetime → 모듈
 - 두 번째 datetime → 모듈 안의 datetime 클래스
 - now() → 현재 시간을 가져오는 함수

```
from datetime import datetime

now = datetime.now()

print(now)
print(now.year)
print(now.month)
print(now.day)
```

Python 기본 모듈

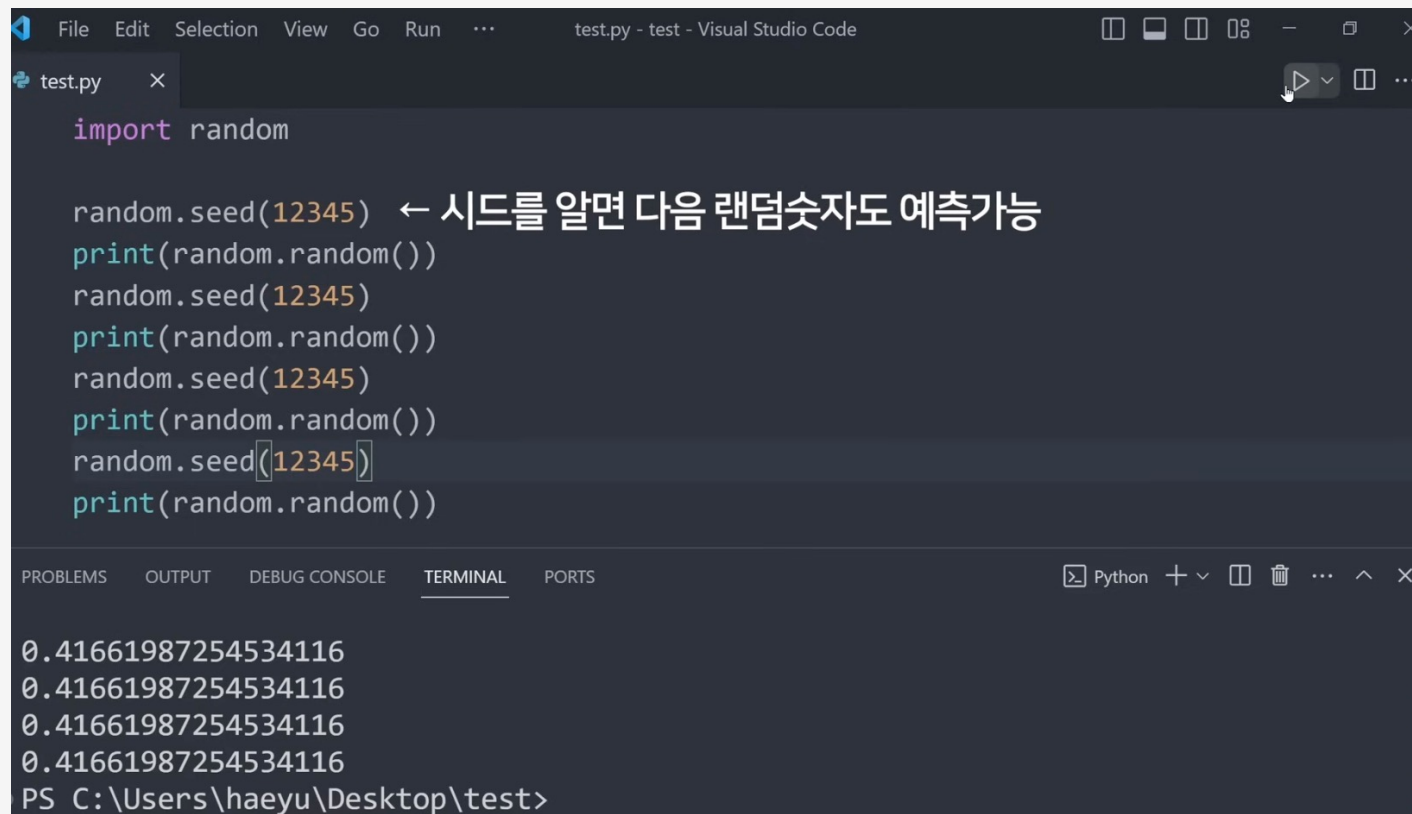
- random
 - randint(1, 10) : 1부터 10 사이의 정수를 랜덤으로 선정
 - choice() : 리스트에서 하나를 랜덤으로 선택

```
import random

print(random.randint(1, 10))
print(random.choice(["apple", "banana", "orange"]))
```

Python 기본 모듈

- random 이 정말 랜덤할까?



The screenshot shows a Visual Studio Code window with a file named 'test.py'. The code in the editor is as follows:

```
import random

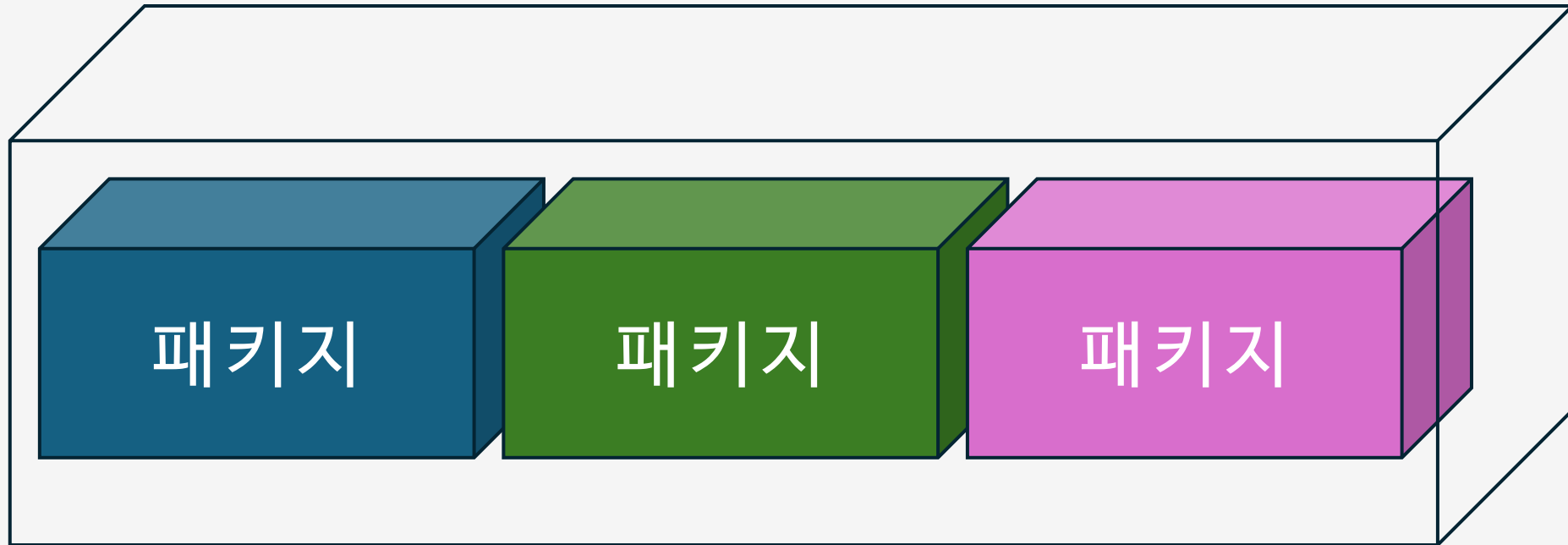
random.seed(12345) ← 시드를 알면 다음 랜덤숫자도 예측가능
print(random.random())
random.seed(12345)
print(random.random())
random.seed(12345)
print(random.random())
random.seed(12345)
print(random.random())
```

The terminal at the bottom shows the output of the script, which consists of four identical floating-point numbers: 0.41661987254534116. The prompt 'PS C:\Users\haeyu\Desktop\test>' is visible at the bottom of the terminal.

2-3 패키지

패키지

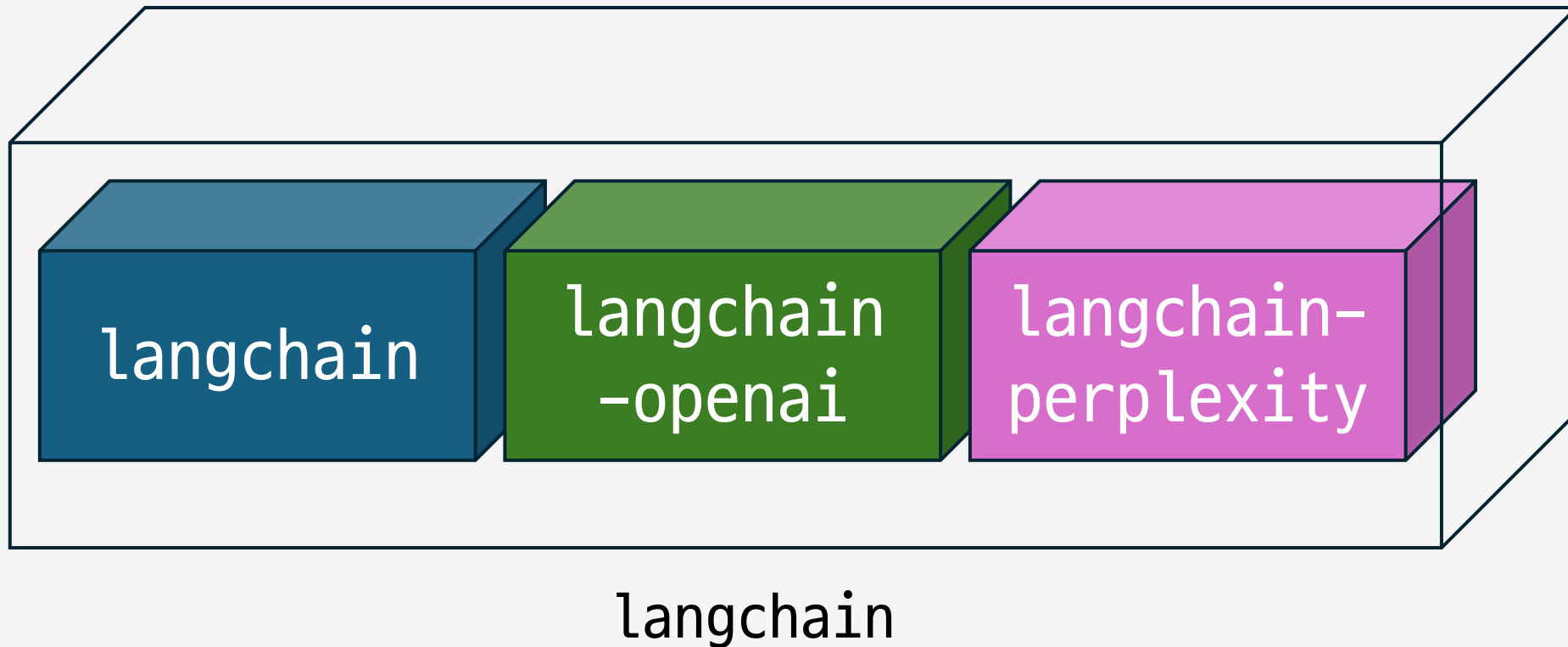
- 패키지 : 모듈(파이썬 파일)을 모아 둔 폴더
- 라이브러리 : 패키지의 묶음



라이브러리

패키지

- <https://github.com/langchain-ai>



패키지

- 모듈(파이썬 파일)을 모아 둔 폴더
 - 프로젝트가 커지면 모듈 파일도 많아 짐 → 파일이 많아지면 한 폴더에 전부 두기 어려움
 - 관련 있는 모듈끼리 폴더로 묶어서 관리!

```
main.py
calculator.py
file_utils.py
api_utils.py
prompt_utils.py
llm_client.py
```



```
project/
├── main.py
├── utils/
│   ├── __init__.py
│   ├── calculator.py
│   └── file_utils.py
└── agent/
    ├── __init__.py
    ├── prompt_utils.py
    └── llm_client.py
```

패키지

- 패키지 생성 실습

```
project/  
├── main.py  
└── utils/  
    ├── __init__.py  
    ├── calculator.py  
    └── text_utils.py
```

```
# utils/text_utils.py  
def make_upper(text):  
    return text.upper()
```

```
def make_lower(text):  
    return text.lower()
```

```
# utils/calculator.py  
def add(a, b):  
    return a + b
```

```
def subtract(a, b):  
    return a - b
```

```
# main.py  
import utils.calculator  
  
result = utils.calculator.add(10, 20)  
  
print(result)
```

패키지

- `__init__.py`
 - 패키지 폴더 안에 들어가는 특별한 파일 (이 폴더를 Python 패키지로 인식하게 함)
 - 비어 있어도 된다
 - 패키지를 불러올 때 실행되는 **초기화 파일**로 사용할 수 있음
 - Python 3.3 이후부터는 `__init__.py` 이 없어도 패키지처럼 동작하기는 함

패키지

- `main.py`
 - `utils` 패키지 안에 있는 `calculator` 모듈을 불러오기

```
# main.py
import utils.calculator

result = utils.calculator.add(10, 20)

print(result)
```

```
# main.py
from utils import calculator

result = calculator.add(10, 20)

print(result)
```

패키지

- `main.py`
 - `utils` 패키지 안에 있는 `calculator` 모듈 중 `add` 함수만 불러오기

```
from utils.calculator import add  
  
result = add(10, 20)  
  
print(result)
```

패키지

- 외부 패키지
 - 직접 만든 패키지 외에도, 다른 개발자들이 만들어둔 패키지를 설치해 사용 가능
 - requests : 웹 요청
 - pandas : 데이터 분석
 - numpy : 수치 계산
 - openai : OpenAI API 사용
 - streamlit : 웹 앱 만들기
 - `pip install requests`
 - pip : 패키지를 관리하는 관리자

패키지

- 외부 패키지
 - langchain 패키지 설치 (https://github.com/langchain-ai/langchain/tree/master/libs/langchain_v1/langchain)

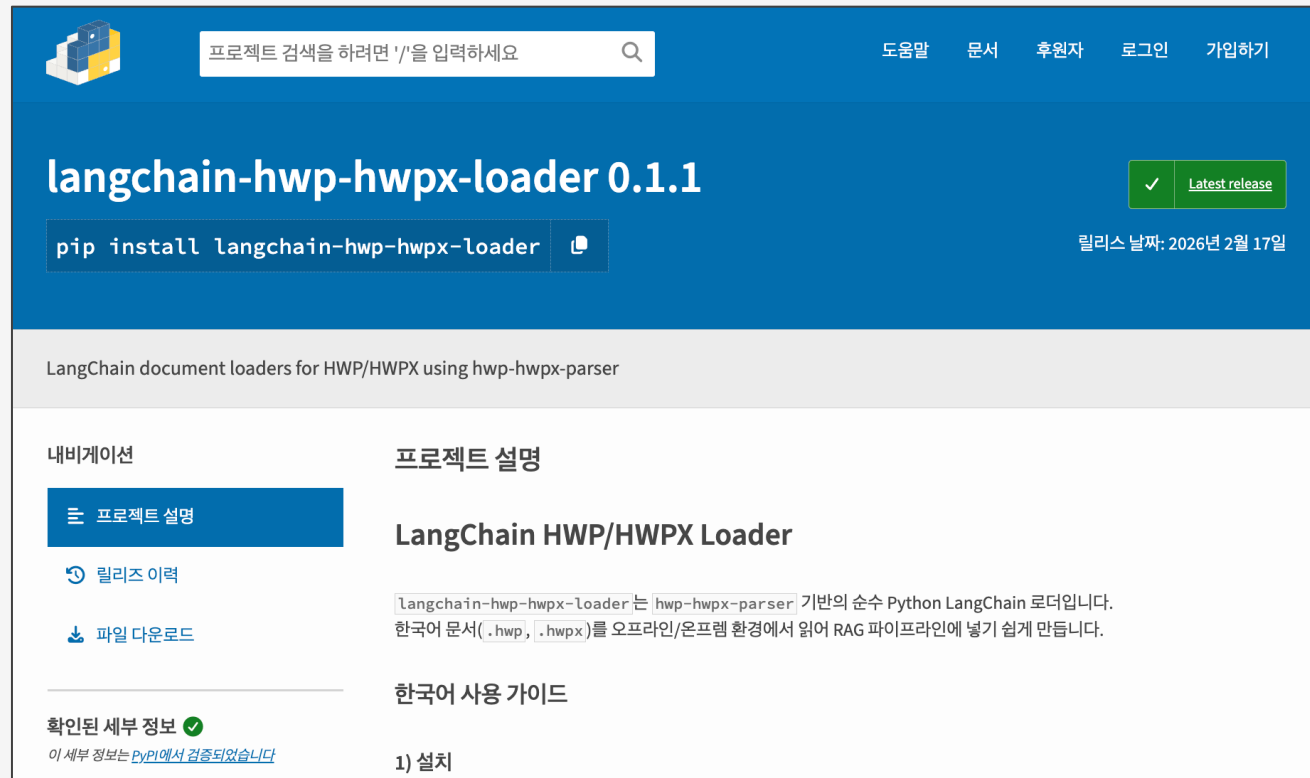
```
pip install langchain
```

- langchain 패키지 속 chat_models 모듈 중 init_chat_model 클래스 import
(https://github.com/langchain-ai/langchain/tree/master/libs/langchain_v1/langchain/chat_models)

```
from langchain.chat_models import init_chat_model  
  
model = init_chat_model("gpt-5.4-mini")
```


패키지

- 외부 패키지
 - <https://pypi.org/project/langchain-hwp-hwp-x-loader/>



The screenshot shows the PyPI page for the package `langchain-hwp-hwp-x-loader` version 0.1.1. The page has a blue header with a search bar and navigation links. The main content area is white with a blue sidebar on the left. The package name and version are prominently displayed at the top, along with a green 'Latest release' badge. Below this, the installation command `pip install langchain-hwp-hwp-x-loader` is shown. The package description states it is a LangChain document loader for HWP/HWPX files using the `hwp-hwp-x-parser` library. The sidebar contains links to the project description, release history, and file downloads. The main content area also includes a section for the project description and a link to the Korean usage guide.

프로젝트 검색을 하려면 '/'을 입력하세요

도움말 문서 후원자 로그인 가입하기

langchain-hwp-hwp-x-loader 0.1.1

✓ Latest release

릴리스 날짜: 2026년 2월 17일

```
pip install langchain-hwp-hwp-x-loader
```

LangChain document loaders for HWP/HWPX using hwp-hwp-x-parser

내비게이션

- 프로젝트 설명
- 릴리즈 이력
- 파일 다운로드

프로젝트 설명

LangChain HWP/HWPX Loader

`langchain-hwp-hwp-x-loader`는 `hwp-hwp-x-parser` 기반의 순수 Python LangChain 로더입니다. 한국어 문서(`.hwp`, `.hwp-x`)를 오프라인/온프레임 환경에서 읽어 RAG 파이프라인에 넣기 쉽게 만듭니다.

한국어 사용 가이드

1) 설치

확인된 세부 정보 ✓
이 세부 정보는 [PyPI](#)에서 검증되었습니다

제 4장.

참고 자료

4-1 타입 힌팅

타입 힌팅

- 동적 언어와 정적 언어
 - 동적 언어 : 프로그램 실행 중에 변수의 타입을 동적으로 바꿀 수 있음 (파이썬)

```
a = 1
print(type(a)) # <class 'int'>

a = "1"
print(type(a)) # <class 'str'>
```

- 정적 언어 : 한 번 변수에 타입을 지정하면 지정한 타입 외의 다른 타입은 사용할 수 없음 (자바)

```
int a = 1; // a 변수를 int형으로 지정
a = "1";   // a 변수에 문자열을 대입할 수 없으므로 컴파일 에러
```

타입 힌팅

- 동적 언어의 장단점
 - 타입에 자유로워 유연한 코딩이 가능하고, 쉽고 빠르게 프로그램을 만들 수 있음
 - 타입 체크를 위한 코드가 없어 비교적 깔끔한 소스 코드를 작성할 수 있음
 - 프로젝트 규모가 커질수록 타입을 잘못 사용하여 버그가 생길 확률도 높아짐
- 👉 안전성을 선호하는 금융권 프로젝트에서는 이런 이유로
동적 언어보다 정적 언어를 주요 언어로 선택하는 경향이 있음

타입 힌팅

- 타입 어노테이션 (type annotation)
 - 동적 언어의 단점을 극복하기 위해 Python 3.5 버전부터 타입 어노테이션 기능 지원
 - 단, 강제성이 있진 않고, 힌트만 제공
 - 변수 뒤에 : 타입 입력

```
num: int = 1  
name: str = "홍길동"  
numbers: list = [1, 2, 3]
```

타입 힌팅

- 함수에 타입 지정하기

```
def add(a: int, b: int) -> int:  
    return a + b  
  
def greet(name: str) -> str:  
    return f"안녕하세요, {name}님!"  
  
def get_user_info(user_id: int) -> dict:  
    return {"id": user_id, "name": "홍길동"}
```

타입 힌팅

- typing 모듈
 - 더 구체적인 타입 정보를 작성하기 위한 Python 내장 모듈

```
from typing import List, Dict

languages: List[str] = ["python", "java", "c++"]

user_info: Dict[str, str] = {"name": "Alice", "age": "30"}

users: List[Dict[str, str]] = [
    {"name": "Bob", "age": "25"},
    {"name": "Charlie", "age": "35"}
]
```


타입 힌팅

- typing 모듈
 - Optional : 값이 있을 수도 있고, 없는 경우

```
from typing import Optional

def find_user(name: str) -> Optional[dict]:
    users = [
        {"name": "Alice", "age": 30},
        {"name": "Bob", "age": 25}
    ]

    for user in users:
        if user["name"] == name:
            return user
    return None

result = find_user("Bob")
print(result) # {'name': 'Bob', 'age': 25}

result = find_user("Jay")
print(result) # None
```

타입 힌팅

- typing 모듈
 - Any : 타입을 정확히 알 수 없거나, 어떤 값이든 받을 수 있도록 하는 경우

```
from typing import Any

def print_value(value: Any) -> None:
    print(value)

print_value("Hello, World!")
print_value(42)
print_value([1, 2, 3])
```

4-2 데코레이터

데코레이터

```
from langchain.tools import tool
```

```
@tool
```

```
def get_weather(city: str) -> str:  
    """Get weather for a city."""  
    return f"It is currently sunny in {city}."
```

```
@overload
```

```
def init_chat_model(  
    model: None = None,  
    *,  
    model_provider: str | None = None,  
    configurable_fields: None = None,  
    config_prefix: str | None = None,  
    **kwargs: Any,  
) -> _ConfigurableModel: ...
```

@ : 데코레이터

함수 빼돌리기

데코레이터

- 함수의 특징 1
 - 함수를 값처럼 사용할 수 있다.
 - 함수 객체를 key에 대한 value로 사용

```
def add(a,b):  
    return a + b  
  
def sub(a,b):  
    return a - b  
  
calc_dict = {  
    "덧셈" : add,  
    "뺄셈" : sub  
}  
  
result = calc_dict["덧셈"](4,3)  
  
print(result) # 7
```

데코레이터

- 함수의 특징 2
 - 함수를 함수의 인자로 사용할 수 있다.
 - add 함수를 calc함수의 인자로 사용할

```
def calc(func,a,b):  
    return func(a,b)  
  
def add(a,b):  
    return a + b  
  
result = calc(add, 4, 3)  
  
print(result) # 7
```

데코레이터

- 함수의 특징
 - 함수를 값처럼 사용할 수 있다.
 - 함수를 함수의 인자로 사용할 수 있다.

```
def add(a,b):  
    return a + b  
  
def deco(func):  
    print('deco')  
    return func  
  
add_with_deco = deco(add)  
  
print(add_with_deco(4,3))  
# deco  
# 7
```


데코레이터

- 데코레이터 : 함수 빼돌리기

```
def add(a,b):  
    return a + b  
  
def deco(func):  
    print('deco')  
    return func  
  
add_with_deco = deco(add)  
  
print(add_with_deco(4,3))  
# deco  
# 7
```

```
def deco(func):  
    print('deco')  
    return func  
  
@deco  
def add(a, b):  
    return a + b  
  
result = add(4,3)  
  
print(result)  
# deco  
# 7
```

데코레이터

- 데코레이터 사용 목적
 - 귀찮은 반복을 줄여줌
 - 원본 코드가 깔끔해짐

```
def add(a, b):  
    print("--- 계산을 시작합니다 ---")  
    result = a + b  
    print("--- 계산이 끝났습니다 ---")  
    return result
```

```
def multiply(a, b):  
    print("--- 계산을 시작합니다 ---")  
    result = a * b  
    print("--- 계산이 끝났습니다 ---")  
    return result
```

```
print(add(4, 3))  
print(multiply(4, 3))
```



```
def calc_notifier(func):  
    def wrapper(a, b):  
        print("--- 계산을 시작합니다 ---")  
        result = func(a, b)  
        print("--- 계산이 끝났습니다 ---")  
        return result  
    return wrapper
```

```
@calc_notifier  
def add(a, b):  
    return a + b
```

```
@calc_notifier  
def multiply(a, b):  
    return a * b
```

```
print(add(4, 3))  
print(multiply(4, 3))
```

데코레이터

- 다양한 데코레이터 문법

↓ 어떻게 될까?

```
46 @nn.compact
47 def predict_length(self,
48     """Predicts the length
```



파이썬 - decorator

임커밋 · 재생목록

decorator 문법? " 빼돌리기 " · 3:19

@을 함수처럼 쓴다면? · 3:19

전체 재생목록 보기

동영상 6개

수고하셨습니다.

